EVMと日時実績ログを用いた個人タスク進捗モデルの提案と実装

明星大学情報学部情報学科
長研究室
22J5-114 野々村空河

2025年12月9日

目 次

1 はじめに **1**

2 研究の目的 **1**

3 関連研究 **2**

4 システムの実装 **4**

4.1 開発環境 4
4.2 システム構成 4
4.3 データモデル 5
4.4 主要機能の実装 7
4.4.1 タスク登録（TodoInput.jsx） 7
4.4.2 実績入力（ProgressEntry.jsx） 8
4.4.3 進捗指標計算（updateStats.js） 8
4.4.4 リスクレベル判定（SPI 基準） 9
4.4.5 当日の作業計画におけるタスク選定アルゴリズム 9
4.4.6 推奨作業時間（todayMinutes）の算出 10
4.4.7 今日のプラン再計算機能 11
4.4.8 通知機能（Cloud Functions） 12
4.4.9 ログ編集機能（LogEditorModal） 12
4.4.10 データエクスポート機能 12
4.5 UI 画面の構成 12
4.5.1 カレンダー画面 13
4.5.2 タスク一覧画面 13
4.5.3 分析画面 13
4.5.4 ログ編集画面 15
4.5.5 設定画面 15
4.5.6 今日のプラン画面 16

5 評価・実験 **17**

5.1 目的 18
5.2 方法と対象 18
5.3 ログの分析方針と可視化項目 19
5.4 リスクと限界への備え 19

6 結果・考察 **19**

7 まとめ **19**

1　はじめに

長期的な学習タスクや継続的なプロジェクトにおいて，日々の進捗を正確に把握し，適切なタイミングで行動を修正することは容易ではない。典型的な ToDo アプリはタスクの登録と締切管理を中心に構成されており，進捗が計画からどの程度逸脱しているのか，あるいは現状のペースで間に合うのかといった情報を十分に提供しない。この結果，利用者が遅れに気づくのは締切が迫ってからであり，必要な行動修正が後手に回るという問題が生じやすい。

このような課題は，利用者が自分自身の作業状況を適切に理解し，自律的に調整する「自己調整（Self-Regulation）」の難しさに起因すると考えられる。自己調整には，現状把握，計画との乖離の認知，改善行動の実施といった循環が必要であるが，従来のタスク管理ツールではこの循環を支援する情報設計が十分とは言えない。

一方，プロジェクト管理分野では Earned Value Management（EVM）が広く利用されており，実績データに基づいて進捗を定量評価し，遅延の早期検知や完了見込みの予測を行う体系が確立されている。特に，進捗効率指数である SPI（Schedule Performance Index）や，現在のペースを維持した場合の完了予測日である EAC（Estimate at Completion）は，計画と実績の乖離を直感的に把握する上で有用である。

しかしながら，従来の EVM は企業プロジェクト向けに設計されており，個人の学習タスクのように作業量が日々変動し，計画も固定的でない状況にそのまま適用することは難しい。個人タスクでは実績データが日次単位で変化し，継続的な再計算と柔軟なフィードバックが不可欠となる。

そこで本研究では，EVM の考え方を個人タスク管理向けに再構成し，日次実績ログに基づいて進捗指標を自動計算する「個人タスク進捗管理モデル」を提案する。本モデルでは，

- pace7d（直近 7 日間の平均実績ペース）
- SPI（進捗効率指数）
- EAC（現在のペースを維持した場合の予測完了日）

といった指標をタスクごとに継続的に計算し，遅延リスクの早期検知，必要ペースの明示，進捗理解の促進を実現する。特に，日々の実績記録に応じてすべての指標が自動更新される仕組みを構築することで，利用者は「今どれだけ進んでいるか」「このペースで間に合うか」を直感的に把握でき，適切な行動修正が可能となる。

本研究では，このモデルを実装した進捗マネジメントアプリを開発し，実際の利用ログを用いて，遅延の早期検知・行動修正・理解促進といった観点から提案モデルの有効性を評価する。

2　研究の目的

本研究の目的は，日次実績ログに基づいて EVM 指標（pace7d, SPI, EAC）を連続的に再計算し，個人の長期タスクに対して「遅延の早期検知」「行動修正」「進捗理解の促進」を実現する進捗管理モデルを構築することである。

従来の ToDo アプリはタスクの登録と締切管理に重点が置かれ，利用者が現在どの程度計画から乖離しているのか，また現状のペースで締切に間に合うのかを判断する手がかりが乏しい。特に，長期的な学習タスクでは実績が日々変動しやすく，計画と実績のズレに気づくのが遅れやすいという課題がある。

本研究では，この課題に対して以下の 4 点を主要な目的とする。

- **遅延リスクの早期検知**：pace7d と requiredPaceAdj の比較から SPI を算出し，締切超過の兆候を早い段階で把握できる仕組みを提供する。
- **行動修正の支援**：必要ペースと実績ペースの差分を明示し，利用者が学習量を調整するための判断を行いやすくする。
- **進捗状態の理解促進**：EAC（予測完了日）を日次で更新して履歴として可視化することで，行動が進捗予測にどのように影響したかを把握しやすくする。
- **記録習慣と自己調整の形成**：実績入力リマインドや進捗可視化によって，自己調整（Self-Regulation）サイクルの継続を支援する。

これらを通じて，本研究は「計画と実績の乖離を適切なタイミングで認知し，行動を修正できる個人タスク進捗管理モデル」の構築を目指す。

3 関連研究

近年、個人向けのタスク管理サービスは数多く提供されており、Google ToDo、TickTick、Notion、Trello などが広く利用されている。これらのサービスは、タスクの登録、締切設定、通知といった基本機能により、ユーザーの作業整理を支援している。

一方で、多くのサービスは「進捗の定量的な把握」や「見積もりと実績の比較」に基づく支援までは踏み込んでいない。たとえば TickTick はポモドーロタイマーによる作業ログ記録を提供しているが、見積時間 E との比較や、進捗効率指数（SPI）や予測完了日（EAC）といった指標を用いたフィードバックには対応していない。また Notion や Trello のようなデータベース型サービスでは柔軟な管理が可能であるものの、進捗の自動評価やペース分析といった機能は提供されていない。

表 1 に代表的サービスと本研究システムを比較する。ただし、本研究システムも未実装機能を含むため、比較が過度に本研究に有利にならないよう配慮した。

表 1: 主要タスク管理サービスと本研究システムの機能比較

機能項目	Google ToDo	TickTick	Notion	Trello	本研究
タスク登録・締切設定	○	○	○	○	○
見積時間（E）の設定	×	△	△	×	○
実績時間（A）の記録	×	○	△	×	○
SPI/EAC の表示	×	×	×	×	○
pace7d に基づく評価	×	×	×	×	○
作業提案通知	×	△	×	×	△
コラボレーション	×	○	○	○	×
スマホ UI 対応	○	○	○	○	△（PC 特化）

学習者の自己調整と進捗把握の困難性

学習科学の分野では、学習者が自らの進捗を正確に把握することの難しさが多数指摘されている。野上ら [1] は、学習者が自身の学習量を過大・過小に評価する傾向を報告しており、主観的な

自己評価だけに依存した学習管理の限界を示している。また、学習時間や作業ログといった客観的データの可視化は、学習行動の調整（self-regulation）を促進することも示されている。これらの知見は、本研究が pace7d や SPI、EAC といった客観的指標を日次ログから算出する意義を裏付けるものである。

自己調整学習モデルと行動フィードバック

Kanfer (1990) は産業・組織心理学の観点から、セルフモニタリング（自己観察）と自己調整学習（self-regulation）の関係を整理し、行動データの継続記録とフィードバックが目標達成能力の向上に寄与することを示している。本研究が日次ログ記録と EVM 指標を組み合わせてフィードバック UI を提供する設計は、こうした理論と一致している。

作業時間見積もりの誤差に関する研究

作業時間の見積もりに関しては、**Connolly and Dean (1997)** がソフトウェア開発において見積時間が体系的に過小評価されることを示し、「楽観バイアス」や「過度な自信」による誤差の発生を指摘している。ただし、本研究では「見積精度の向上」を主要目的とはしておらず、この知見は主に「見積 E と実績 A の乖離が自然に起こりうる」という背景情報として位置づける。

EVM の個人タスクへの応用と課題

Earned Value Management（EVM）は、プロジェクト管理において進捗を定量評価する代表的手法であり、SPI（Schedule Performance Index）や EAC（Estimate at Completion）を用いて計画と実績のズレを評価できる。しかし、従来の EVM は企業プロジェクト向けに最適化されており、個人の学習タスクのように日々実績が変動する文脈ではそのまま適用できない。具体的には、

- PV（計画値）が個人タスクの柔軟な計画に適合しづらい
- 実績ログが日ごとに変化し、SPI や EAC の頻繁な再計算が必要
- 完了見込みの変化が短期間で大きく変動しうる

といった課題が存在する。

実務で用いられる基準として、PMI や NASA、米国国防総省（DoD）の EVM Implementation Guide では、一般に SPI ≥ 1.0 を計画通り、0.8–1.0 を軽度の遅延、0.8 未満を重大な遅延とする例が多く、本研究のリスク分類の判断基準として参考にしている。

本研究の位置づけ

以上のように、既存のタスク管理サービスでは進捗の自動評価や行動修正支援が十分ではなく、また EVM の個人タスクへの応用も限定的である。本研究は、EVM 指標（pace7d・SPI・EAC）を日次ログと組み合わせ、遅延の早期検知や行動修正を支援する個人向け進捗管理モデルを構築する点で、従来研究の空白を埋める意義を有する。

4　システムの実装

本章では、本研究で開発した進捗マネジメントアプリの構成と実装について述べる。まず開発環境を示し、次にシステム全体の構造、データモデル、主要機能、UI 設計について順に説明する。

4.1　開発環境

本研究のアプリケーションは，Web ブラウザ上で動作するシングルページアプリケーション（SPA）として実装した。主な使用技術は以下のとおりである。

- フロントエンド：React（Vite）
- 言語：JavaScript / JSX
- バックエンド：Firebase（Authentication, Firestore, Cloud Functions）
- 通知：Firebase Cloud Messaging（FCM）
- 実行環境：Node.js
- バージョン管理：Git / GitHub

Firebase は認証・データベース・バックエンド処理（Functions）・通知（FCM）を統合的に提供しており，進捗計算やログ集計のリアルタイム更新と相性が良いため採用した。

4.2　システム構成

本システムは、React を用いたフロントエンド、Firestore を中心とするデータベース層、そして Google Cloud Functions を用いたバックエンド処理の 3 層構成で動作する。図 1 に全体構成を示す。

ユーザー操作はフロントエンドで行われ、データは Firestore に保存される。Firestore の更新をトリガとして Cloud Functions が進捗指標の計算や通知スケジューリングを自動実行することで、ユーザーの入力に応じて常に最新状態の進捗データが維持される。

フロントエンドでは、タスク登録や実績入力、分析画面での可視化といったユーザーとのインタラクションが中心となる。タスク一覧画面には、EVM 指標（SPI・EAC・残作業量など）がリアルタイムに反映され、各タスクの遅延リスクは SPI に基づいて「良好」「注意」「遅延」に色分けされて表示される。

一方、Firestore ではタスクデータ・日次実績ログ・必要ペース（requiredPace）・SPI・EAC といった EVM 指標を保持する。日次ログは actualLogs として日付キーで管理され、後述する pace7d の計算に利用される。

Cloud Functions では Firestore の変更を監視し、以下の主要処理を自動実行する。

- updateStats.js：日次実績ログが更新されるたびに、pace7d、残作業量、必要ペース、SPI、EAC を再計算し Firestore に書き戻す。ログ編集にも対応しており、過去の実績が修正された場合でも全期間の SPI および EAC を整合的に再計算する。
- scheduleMorningSummary.js：毎朝、タスクの優先度や遅延リスクを SPI に基づいて評価し、当日の推奨タスクを通知する。

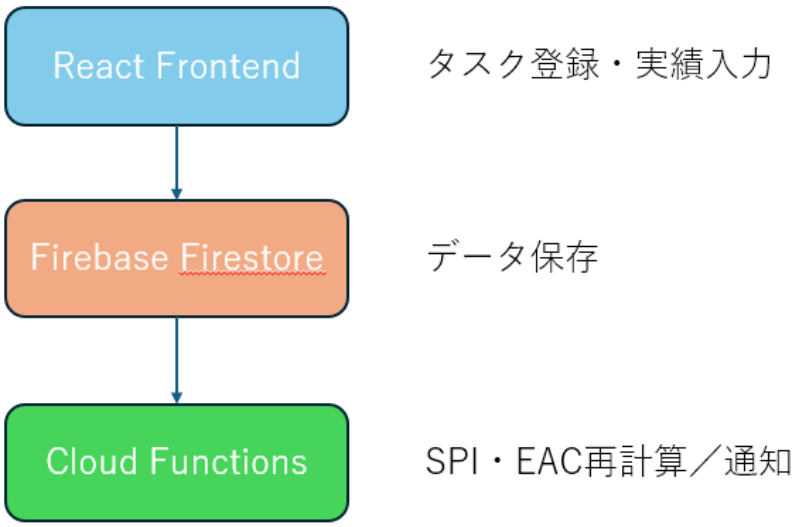


図 1: 本研究システムの全体構成

- scheduleProgressReminder.js：夜間に実績未入力を検知し、記録を促すリマインド通知を送信する。
- risk monitoring（内部ロジック）：SPI の低下や EAC の締切超過を検出すると、必要に応じて遅延リスク通知を送信する。リスク判定には PMBOK や NASA/DoD の基準に基づく SPI 閾値を採用する。

この 3 層構成により、ユーザーが実績入力を行うだけで EVM 指標の自動更新・通知処理・可視化が連鎖的に行われ、進捗管理の循環（記録 → 評価 → フィードバック）が自動的に機能するアプリケーション構造が実現されている。

4.3 データモデル

本研究のアプリケーションは，タスク情報・日次実績ログ・EVM 指標を総合的に管理するために，Firestore を用いた階層型データモデルを採用している。図 2 に，本アプリで用いる主要データ構造の概要を示す。本モデルは，ユーザーが実績を入力するたびに Cloud Functions が自動的に再計算を行い，最新の進捗指標が常に Firestore 上に反映されることを前提として設計している。

本研究のアプリでは，進捗管理に必要な情報を以下のように保持する。

- **todos コレクション（タスク）**
  - text：タスク名
  - estimatedMinutes（E）：見積時間
  - actualTotalMinutes（A）：累積実績
  - actualLogs：日次実績ログ（YYYY-MM-DD: 分）
  - pace7d：直近 7 日間の平均実績ペース

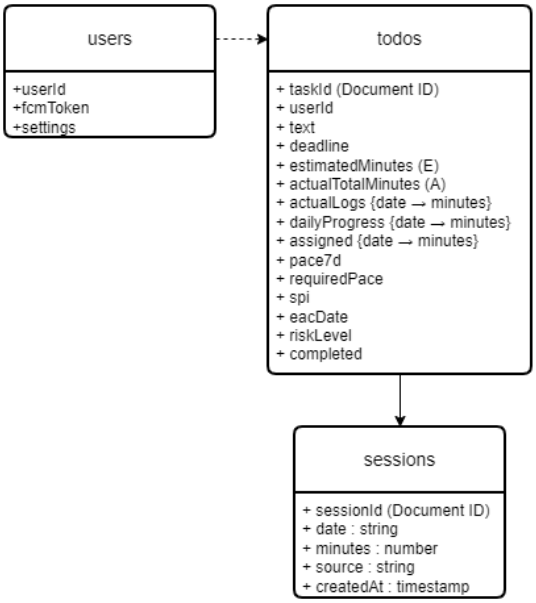


図 2: Firestore のデータモデル（タスク・実績ログ）

  - requiredPaceAdj：期日までに完了するための必要ペース
  - spi：進捗効率指数（SPI）
  - eacDate：予測完了日（EAC）
  - deadline：締切日時
  - completed：完了フラグ
  - createdAt：タスク登録日時
  - lastProgressAt：最後に実績が記録された日時
  - userId：タスク所有ユーザ ID

- **sessions サブコレクション**
  - date：実績入力日
  - minutes：入力された作業時間
  - source：入力経路（手動 / 通知から）
  - createdAt：記録時刻
- **users コレクション**
  - fcmToken：通知送信に使用するトークン
  - settings：通知 ON/OFF などの設定
  - loginCount：利用回数（継続性の指標）

特に actualLogs と pace7d は本研究特有のデータ構造であり，EVM 指標を日次ベースで再計算するための基盤となっている。

表 2: todos コレクションの主なフィールド（改訂版）

フィールド名	概要
text	タスク内容（例：IT パスポート 100 時間勉強する）。
deadline	タスク全体の締切日時 D。
estimatedMinutes	見積総学習時間 E［分］。進捗率・残作業量算出の基礎となる。
actualLogs	日付ごとの実績時間 $A(t)$ を保存するマップ。pace7d，EAC，SPI の再計算に使用。
actualTotalMinutes	累積実績時間 $\sum_t A(t)$［分］。
dailyProgress	日別の累積進捗率（その日までの EV ／ 総 EV）。
actualProgress	現時点の累積進捗率（EV ／ 総 EV）。
idealProgress	線形計画に基づく理想進捗率。計画との差分（遅れ判定）に利用。
pace7d	直近 7 日間の平均学習ペース［分／日］。SPI・EAC の基礎指標。
requiredPace	遅れを考慮しない基本の必要ペース（残作業量 ÷ 残日数）。
requiredPaceAdj	遅れ状況（idealProgress と actualProgress の差）を考慮し，遅れ量を残日数で均等に取り戻す前提で算出される調整後必要ペース［分／日］。
spi	SPI（進捗効率指数：pace7d ／ requiredPaceAdj）。
spi7d	直近 7 日間の SPI。pace7d と requiredPaceAdj から算出される。
eacDate	現在のペースを続けた場合の予測完了日（EAC）。
riskLevel	遅延リスクのカテゴリ（“ok”, “warn”, “danger” など）。
completed	タスク完了フラグ。
createdAt	タスク登録日時。
lastProgressAt	最後に実績が記録された日時。
userId	タスク所有ユーザ ID。
eacHistory	日次で再計算された EAC の履歴を保持し，進捗推移分析に利用する。

4.4 主要機能の実装

本節では，本研究で開発した進捗マネジメントアプリの主要機能について述べる。システムは「記録された実績をもとに EVM 指標を自動計算し，早期に補正すべきタスクを提示する」という研究目的を実現するため，タスク登録，実績入力，進捗指標計算，リスク判定，通知処理，ログ編集，データエクスポートといった一連の処理を統合的に設計している。これらの処理はすべて Firestore と Cloud Functions により自動化され，ユーザーの入力に連動して最新の進捗状態が維持される。

4.4.1 タスク登録（TodoInput.jsx）

ユーザーがタスク名，締切日時，見積時間（E）を登録すると，Firestore の `todos` コレクションに新規ドキュメントが作成される。登録された見積時間 E は，残作業量 R，必要ペース（requiredPaceAdj），SPI，EAC 等の EVM 指標の基礎データとして用いられる。また，タスクはラベルや作成日時，ユーザー ID とともに保存され，後述する分析機能に利用される。

- **見積作業量 E の確立**：EVM の根幹となる指標（残作業量，必要ペース，SPI，EAC）の計算基盤を構築する。

- **タスクの明確化**：見積値を設定することで，ユーザーはタスク規模を意識しやすくなる。

4.4.2 実績入力（ProgressEntry.jsx）

ユーザーが 1 日の作業時間を入力すると，以下の処理が自動で行われる。

1. `actualLogs[日付]` への実績加算
2. `actualTotalMinutes`（累積実績 A）の更新
3. `sessions` サブコレクションへの入力ログ保存

記録された日次実績は，pace7d（直近 7 日平均）の算出に利用され，EAC および SPI の精度向上に寄与する。また，実績は後から修正可能であり，過去ログの変更にも対応している。

4.4.3 進捗指標計算（updateStats.js）

Firestore の `actualLogs` が変更されるたびに，Cloud Functions によってすべての EVM 指標が再計算される。本研究で使用した主な指標は以下の通りである。

- 残作業量：$R = E - A$
- 直近平均ペース：pace7d（過去 7 日間の平均作業時間）
- 必要ペース：requiredPaceAdj（締切までに必要な 1 日あたり作業量）
- 進捗効率指数：$\mathrm{SPI} = \frac{\mathrm{pace7d}}{\mathrm{requiredPaceAdj}}$
- 予測完了日（EAC）：現在の pace7d を継続した場合の完了見込み

EAC 予測日の履歴更新：
本研究では EAC を単なる最新値として保持するのではなく，日次の実績ペースに応じて変化する EAC を「履歴」として保存するデータ構造 `eacHistory` を導入した。updateStats.js は毎回の再計算の際に，当日付の EAC を `eacHistory[YYYY-MM-DD]` として記録することで，EAC の推移を時系列で蓄積する。

この履歴により,「いつ予測完了日が遅れ始めたか」「改善行動によってどの程度回復したか」を折れ線グラフとして可視化することが可能となる。また，過去の実績ログを修正した場合には，全期間の pace7d，requiredPaceAdj，SPI，EAC を再計算し，`eacHistory` の全日付を整合的に更新するように設計されている。

Cloud Functions は，これらの計算を行った後，結果を Firestore に書き戻す。特に本研究では，過去ログの編集にも対応できるように，全期間の pace7d，SPI，EAC を自動的に再計算する仕組みを導入した。

4.4.4 リスクレベル判定（SPI 基準）

本システムでは，EVM の中心指標である SPI を用いて遅延リスクを分類する。判定基準は PMBOK や NASA/DoD の EVM 運用基準を参考にし，以下の 3 段階に設定した。

- SPI ≥ 1.0：良好（計画通り）
- $0.8 \leq$ SPI < 1.0：注意（僅かな遅延）
- SPI < 0.8：遅延（重大な遅延）

また，EAC が締切を超過する場合や，締切当日を過ぎても完了していない場合には，SPI に関係なく遅延として扱う。判定結果は `riskLevel` として保存され，後述する UI 層で参照される。

4.4.5 当日の作業計画におけるタスク選定アルゴリズム

本システムでは、ユーザーが当日取り組むべきタスクを提示するために、各タスクの進捗状況、締切日、残作業量、および必要ペースを用いて「今日のプラン（推奨タスク）」を生成する。本節では、どのタスクを当日の候補として選定するかについて述べる。

本アルゴリズムは、未完了かつ削除されていないタスクを対象とし、当日の基準日はユーザー端末のローカル時刻により決定される。

(1) 遅れタスクの抽出　対象となる各タスクについて，締切日 D と見積総作業量 E を用いて，タスク作成日から当日までの経過日数 t と総期間 T から理想進捗率

$$\text{ideal} = \frac{t}{T}$$

を計算する。さらに，実績ログから累積実績時間 A を取得し，実進捗率

$$\text{actual} = \frac{A}{E}$$

を算出する。両者の差

$$\text{lag} = \text{ideal} - \text{actual}$$

が正となるタスクを「遅れタスク」として候補集合に含める。

(2) 優先度によるソート　候補タスクは、以下の三つの基準に基づいて優先度を決定し、並べ替える。

1. 遅れ度 lag の大きさ
2. 締切までに必要な 1 日あたり作業量（requiredPaceAdj または requiredPace）
3. 締切日の早さ

ここで用いる requiredPaceAdj は，各タスクに対して Cloud Functions により最新の実績ログをもとに算出される 1 日あたりの必要作業量である（詳細は表 2 を参照）。遅れ度と必要作業量の双方を考慮することで，締切に向けて優先的に取り組むべきタスクを抽出できる。

(3) 上位タスクの選定　優先度順に並べたタスクのうち、上位 3 件を「今日のプラン」として採用する。これらのタスクに対して割り当てる推奨作業量（todayMinutes）は、残作業量・必要ペース・当日のキャパシティを組み合わせた最小値により決定される。

(4) フォールバック処理　遅れタスクが存在しない場合は，未完了タスクを必要ペースの大きさと締切の近さに基づいて並べ替え，同様に上位 3 件を選定する。

(5) まとめ　以上のように、本システムのタスク選定アルゴリズムは、「遅れの大きさ → 必要ペース → 締切」 の順で優先度を決定し、遅れタスクがない場合は必要ペースと締切による代替選択を行う。

本選定処理と、次節で述べる推奨作業量 todayMinutes の割当処理を組み合わせることで、ユーザーに対し、その日に着手すべきタスクと適切な作業量を提示できるようになっている。

4.4.6　推奨作業時間（todayMinutes）の算出

前節で選定した最大 3 件のタスクに対し，本節では当日に割り当てる推奨作業時間 todayMinutes をどのように決定するかを述べる。アプリの「今日のプラン」画面では，ユーザ設定の日次キャパシティ C （settings.app.dailyCap，未設定時は既定値 120 分）を上限として，各タスクの必要ペースと残作業量に基づき todayMinutes を算出する。以下では DailyPlan コンポーネントにおける `mode="initial"`（初期計算）の挙動を対象とする。

(1) 基本ケース：遅れタスクが存在する場合　優先度付きで並べた候補タスク列（4.4.5 節）を先頭から順に処理し，次の変数を準備する。

- R：当該タスクの残作業量（見積総時間 E から累積実績 A を引いた値）
- required：期日までに完了させるための 1 日あたり必要作業量（`requiredPaceAdj` が存在すればそれを，なければ `requiredPace` を用いる）
- C：当日のキャパシティ（分）
- used：すでに他のタスクに割り当て済みの今日の合計作業時間

このとき，各タスクに割り当てる作業時間 need は，

$$\text{remainCap} = \max(0, C - \text{used})$$

$$\text{need} = \min(\text{required}, R, \text{remainCap})$$

と定義する。need ≤ 0 となる場合はそのタスクには割り当てを行わない。正の値が得られた場合，そのタスクの todayMinutes として need を四捨五入して保存し，used に加算する。この処理を優先度順に繰り返し，used $\geq C$ となるか，todayMinutes が設定されたタスク数が 3 件に達した時点で割当てを終了する。

このアルゴリズムにより，(i) 各タスクに対して残作業量 R を超える時間を割り当てず，(ii) 全タスクの todayMinutes の合計が当日のキャパシティ C を超えないよう制約したうえで，遅れの大きいタスクから順に日次の作業量を配分できる。

(2) フォールバック：遅れタスクが 1 件もない場合　前節の遅れ判定により候補タスクが 1 件も得られない場合は，すべての未完了タスクのうち $E > 0, A < E$ を満たすものを対象に，締切日の早さと必要ペース required に基づいて並べ替える。具体的には，締切日の早い順，締切が同じタスク間では required の大きい順にソートし，上位 3 件を当日の候補とする。

フォールバック時の todayMinutes は，キャパシティや残作業量による制約をかけず，各タスクについて

$$\text{todayMinutes} = \text{required}$$

を四捨五入した値をそのまま採用する。この場合，当日の合計値がキャパシティ C を上回ることもあり得るが，そもそも遅れタスクが存在しない状況であり,「締切と必要ペースの観点から優先的に取り組むべきタスク」と「そのタスクに対する 1 日あたりの目安時間」を提示することを主目的とした近似的な割当てと位置づけている。

4.4.7　今日のプラン再計算機能

前節までに述べた todayMinutes の算出アルゴリズム（4.4.5〜4.4.6）は，その時点での実績ログ・見積時間・締切情報に基づいて初回の作業計画を生成するものである。しかし実際の利用状況では，ユーザが提示された作業時間どおりに進めない場合や，推奨されていないタスクに時間を割く場合，あるいは過去の実績ログや見積時間を後から修正する場合がある。このような場面に対応するため，本システムでは「今日のプランの再計算」機能を提供している。

(1) 再計算処理の流れ　ユーザが再計算ボタンを押すと，DailyPlan コンポーネントでは次の手順で再計算を行う。

1. Firestore から未完了タスクを再取得し，削除済みタスクを除外する。
2. 実績入力やログ編集に応じて，Cloud Functions により最新化された `actualTotalMinutes`, `requiredPaceAdj`, SPI, EAC などの指標を含むタスク一覧を取得する。
3. 取得した未完了タスク集合に対して，4.4.5〜4.4.6 で述べた初回計算と同じ `selectTodayPlan(mode = "initial")` を適用し，新しいプラン候補（nextPlan）を生成する。
4. 既存の todayPlan と nextPlan が完全に一致する場合は,「進捗の変更がないため提案内容は変わりません」と通知するのみで更新は行わない。
5. 異なる場合は，nextPlan を Firestore に保存し，直前のプランを `previousPlan` として履歴に保持する。

この処理により，ユーザはいつでも最新の実績を反映した作業計画を取得できる。

(2) 再計算が有効となる利用場面　本機能は，次のような状況で特に有効に働く。

- **提示された作業時間どおりに取り組まなかった場合：**今日の残作業の進み具合に応じて再配分が行われ，その時点で最適化された作業計画が提示される。
- **推奨されていないタスクに時間を使った場合：**実績ログの増加を反映して遅れ状況や必要ペースが変化するため，再計算後には優先順位が入れ替わることがある。

- **過去ログや見積時間を修正した場合：**requiredPaceAdj，実績進捗，EAC などの指標が再算出されるため，その変更を取り込んだ新しい今日のプランが生成される。
- **キャパシティ上限を守りつつ柔軟に調整したい場合：**todayMinutes の割当ては 4.4.6 で述べたように日次キャパシティ C 内で最適化されるため，何度再計算しても実行可能範囲のプランが得られる。

(3) 機能の位置づけ　再計算機能は，新しいアルゴリズムを適用するものではなく，4.4.5〜4.4.6 で述べた todayMinutes の算出処理を「最新の実績およびタスク情報に基づいて再度実行する」ための仕組みである。これにより，ユーザの実際の行動に追従しながら，常に現時点で最適なタスク選択と時間配分を提示できる点に特徴がある。

4.4.8　通知機能（Cloud Functions）

本システムでは，進捗管理習慣を維持するために，Cloud Functions を用いた通知機能を実装している。主な通知は次の 3 種類である。

- 朝の推奨タスク通知：残作業量やリスクレベルに基づき，当日着手を推奨するタスクを提示する。
- 夜の実績入力リマインド通知：その日に実績が記録されていない場合に，入力を促す。
- 遅延リスク通知：SPI の低下や EAC の締切超過を検知した際に，遅延リスクを知らせる。

これらの通知は Firestore の状態変化や時刻スケジューラをトリガとして Cloud Functions が実行し，ユーザー端末の FCM トークンに対して送信される。

4.4.9　ログ編集機能（LogEditorModal）

ユーザーは過去の実績を修正できる。修正後は Cloud Functions が呼び出され，pace7d，requiredPaceAdj，SPI，EAC，riskLevel が再計算される。本研究では，履歴を遡って修正しても指標の整合性が維持されるように，すべての計算を日次ベースで再実行する設計とした。

4.4.10　データエクスポート機能

分析や評価実験のため，記録された実績ログや EVM 指標を CSV 形式でエクスポートできる機能を実装した。これにより，通知反応率や見積誤差の推移など，研究用データを外部で解析することが可能になる。

4.5　UI 画面の構成

本節では，ユーザーが実際に操作する UI 画面の構成について述べる。4.4.8 節で説明した各種処理（EVM 指標の計算，リスク判定，通知処理など）はバックエンドで自動的に行われ，本節ではそれらの結果がどのように画面上で提示され，ユーザーの自己調整行動を支援しているかに焦点を当てる。

本アプリの主要な画面は，カレンダー画面，タスク一覧画面，分析画面，ログ編集画面，設定画面の 5 つである。

本節では，主要な UI を図 3 〜 図 8 に示し，各画面の役割と研究上の意義を述べる。

4.5.1　カレンダー画面

カレンダー画面（図 3）では，月単位でタスクの締切日や実績入力の有無を俯瞰できる。各日付には締切を持つタスクが表示され，締切が集中している期間や，全く作業していない期間が一目で確認できる。

この画面は，ユーザーが長期的な作業スケジュールを把握し,「どの時期に負荷が偏っているか」「どのあたりから着手すべきか」といった判断を行うための手がかりを提供する役割を持つ。

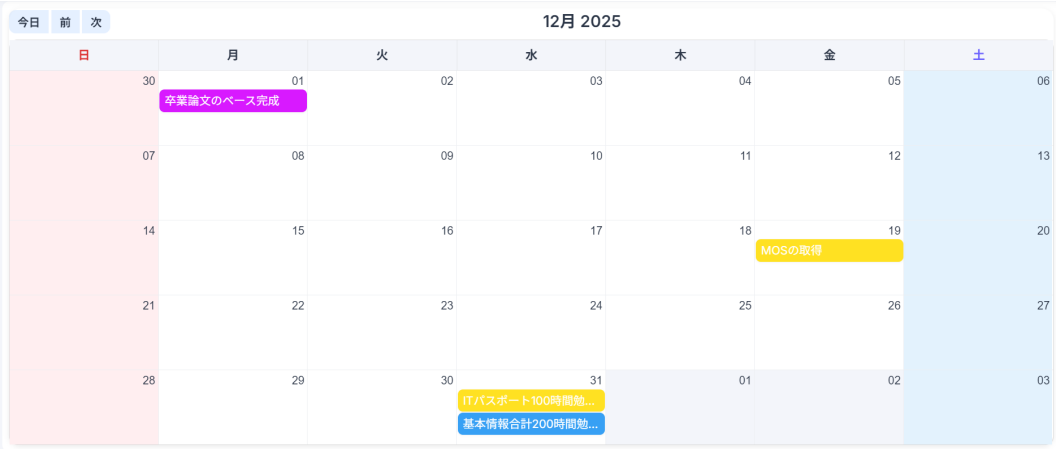


図 3: カレンダー画面（月間のタスク進捗と締切の俯瞰）

4.5.2　タスク一覧画面

タスク一覧画面（図 4）は，本アプリの中心的な画面であり，各タスクの概要と進捗状況が一覧で表示される。各カードには，タスク名，締切，見積時間，累積実績，残作業量に加えて，最新の EVM 指標（pace7d，必要ペース，SPI，EAC）が要約された形で示される。

特に，SPI に基づくリスクレベルは，カードの背景色やバッジとして視覚的に表現される。SPI が 1.0 以上のタスクは緑色（良好），0.8 以上 1.0 未満は黄色（注意），0.8 未満は赤色（遅延）として表示されるため，ユーザーは「どのタスクが危険域に入りつつあるか」を瞬時に把握できる。これにより，着手順序の決定や当日の作業計画に反映しやすくなる。

4.5.3　分析画面

分析画面（図 5）では，EVM 指標を用いた時系列の可視化が行われる。主な表示内容は次のとおりである。

- 日別の作業時間と累積作業量

図 4: タスク一覧画面（SPI, EAC, 残量, 必要ペースなどのリアルタイム可視化）

- 直近 7 日間および 30 日間の平均ペース
- タスク別の SPI 推移（折れ線グラフ）
- 理想的な進捗曲線と実績の比較

これらのグラフにより，ユーザーは「いつからペースが落ち始めたのか」「行動の変化が SPI や EAC にどのように影響したか」を視覚的に理解できる。特に SPI の折れ線グラフは，注意状態や遅延状態に入ったタイミングを振り返る材料となり，自身の学習・作業習慣をメタ認知するきっかけを与える。

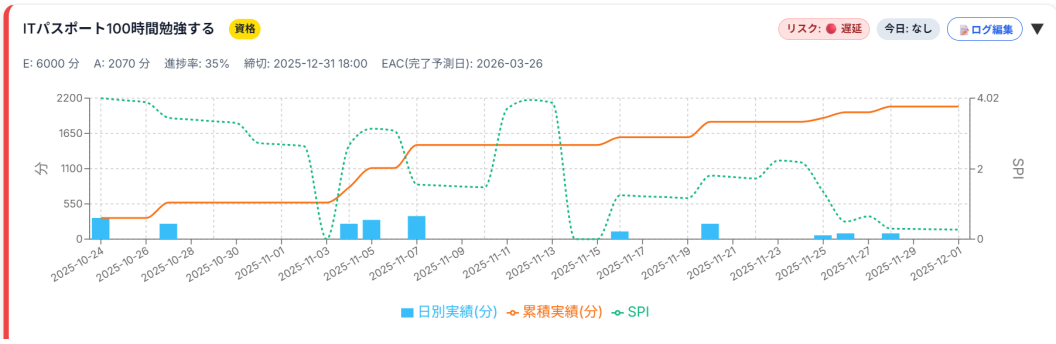


図 5: 分析画面（作業時間推移，直近平均，SPI 推移）

さらに本研究では，タスクごとに日次で再計算される EAC（予測完了日）の履歴を折れ線グラフとして可視化している。このグラフでは，縦軸を 2 つ用い，左軸に残作業量，右軸に予測完了日（EAC）を配置することで,「残作業量の増減」と「予測完了日の遅れ・回復」を同時に把握できるようにした。

これにより，ユーザーは，

- どの時点で EAC が締切を超過し始めたか

- 作業量の増減が EAC にどのように影響したか
- 行動改善（作業ペース向上）が EAC をどの程度回復させたか

を視覚的に確認できる。

図 6 に，本研究で導入した EAC（予測完了日）の日次推移を可視化した折れ線グラフを示す。残作業量（左軸）と EAC 予測日（右軸）を同時に表示することで，実績ペースの変化が締切遅延にどの程度影響しているかを直感的に確認できる。

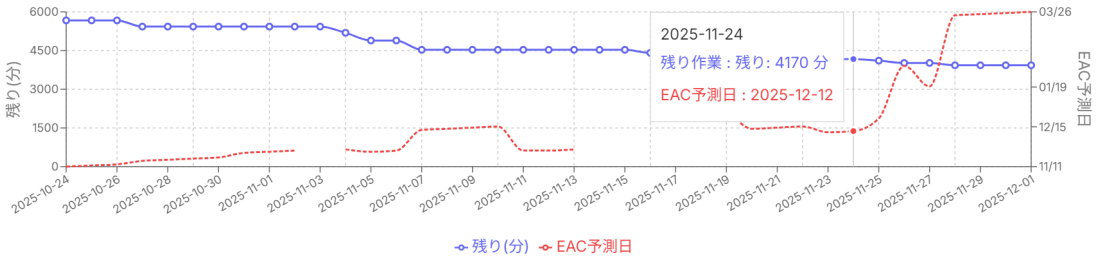


図 6: EAC 予測日の時系列グラフ（残作業量との比較）

一般的なタスク管理では締切遅延の兆候を把握しづらいが、EAC の時系列変化を見ることで「どの行動が遅れの原因となったか」「改善行動がどの程度効いたか」を直接確認でき、これは自己調整学習におけるメタ認知的フィードバックとして重要な役割を果たす。

4.5.4 ログ編集画面

ログ編集画面（図 7）では，日付ごとに記録された実績時間を一覧し，誤入力や記録漏れを修正できる。ユーザーはカレンダーや一覧から日付を選択し，当日の作業時間を増減させることができる。

この画面は，実績データの欠損や誤差を補正し，より現実に近い進捗記録を維持するためのものである。ユーザーにとっては,「ちゃんと記録できていない」という心理的なストレスを軽減し，日次記録の継続を支える役割も持つ。

4.5.5 設定画面

設定画面（図 8）では，通知時刻や通知のオン・オフ，タスクラベルの管理，アプリのバージョン情報などを確認・変更できる。通知時刻を生活リズムに合わせて調整できるようにすることで，ユーザーは無理のないタイミングでリマインドを受け取ることができる。

また，ログイン回数やバージョン情報（コミット ID）を表示することで，利用継続の度合いやシステム更新履歴を簡単に把握できるようにしている。これらは研究上は補助的な情報であるが，ユーザーの利用意欲や再現性の確保に寄与する。

以上のように，本アプリの UI は，4.4 節で述べた EVM 指標計算やリスク判定の結果を，ユーザーが理解しやすい形で提示することを目的として設計されている。各画面はそれぞれ異なる時間スケール（1 日，1 週間，1 か月）や視点（タスク単位，全体の傾向）からフィードバックを与えることで，ユーザーの自己調整を多面的に支援している。

図 7: ログ編集画面（入力忘れや誤入力の修正に対応）

- アプリの通知をユーザーの生活リズムに適応
- ラベル管理によりカテゴリ別学習の分析が可能
- バージョン情報により再現性と信頼性を担保
- ログイン回数の可視化は継続利用の動機づけになる

4.5.6 今日のプラン画面

今日のプラン画面（図 9）では，4.4 節で述べたタスク選定アルゴリズムに基づいて，当日に取り組むべき最大 3 件のタスクが提示される。各タスクカードには

- 残作業量 R
- 必要ペース（requiredPaceAdj）
- 当日割り当てられた推奨作業量（todayMinutes）

が表示され，ユーザーは優先すべきタスクと作業量を直感的に把握できる。

本画面には「今日のプランを更新」ボタンが配置されており，ユーザーが任意のタイミングで当日のプランを再計算できる。再計算時のアルゴリズムは 4.4.7 節で述べたとおり，当日これまでの実績時間（actualToday）を考慮し，残キャパシティに応じた推奨作業量が再算出される。

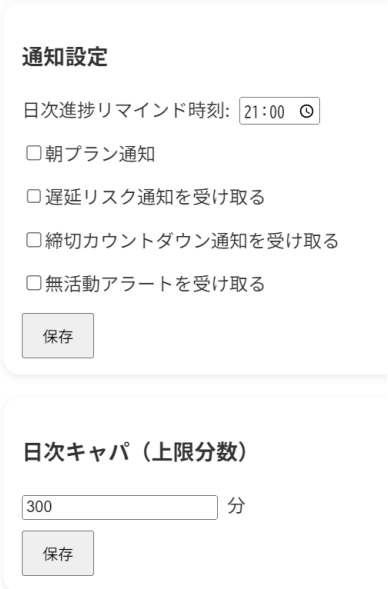


図 8: 設定画面（通知設定，ラベル管理，バージョン情報）

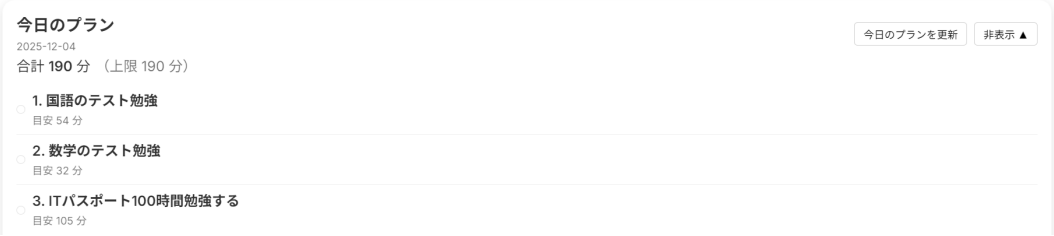


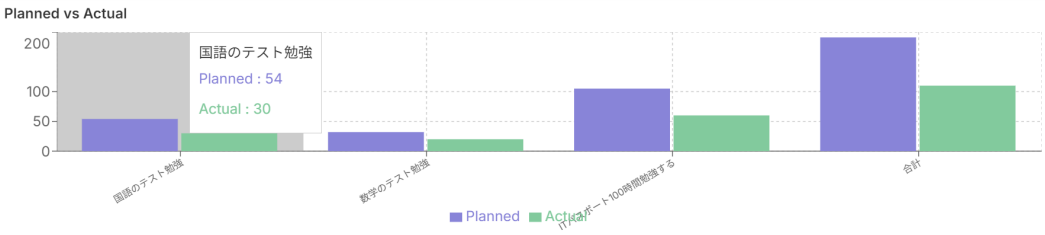
図 9: 今日のプラン画面（タスク一覧と割当作業量の提示）

さらに，図 10 に示すように，本画面では当日の計画値（Planned）と実績値（Actual）が棒グラフによって可視化される。これによりユーザーは，その日の計画達成度を視覚的に確認でき，日中の自己調整行動を支援する役割を果たす。

5　評価・実験

本章では，第 2 章で述べた研究目的（遅延の早期認知，行動修正の支援，進捗理解の促進，記録習慣の形成）が，実際の利用状況においてどの程度達成されているかを検証する。具体的には，提案システムを 2 週間用いて得られた日次ログ・進捗指標（pace7d, SPI, EAC）および通知履歴を分析し，提案モデルがユーザーの自己調整行動に与える影響を定量・定性の両面から評価する。

以下では，まず実験の目的を整理し（5.1 節），続いて方法と対象（5.2 節），分析指標と可視化方針（5.3 節）について述べる。



タスク	予定 (分)	実績 (分)
国語のテスト勉強	54	30
数学のテスト勉強	32	20
ITパスポート100時間勉強する	105	60
合計	**191**	**110**

図 10: 今日のプランに対する計画値（Planned）と実績値（Actual）の可視化

5.1 目的

本実験の目的は，第 2 章で述べた研究目的が実際の利用状況においてどの程度達成されているかを検証することである。具体的には，提案システムが日次実績ログから算出する進捗指標（pace7d, SPI, EAC）が，以下の 4 点に対してどのように寄与するかを評価する。

1. **遅延リスクの早期認知の支援：**SPI の低下や EAC の締切超過などの変化が，利用者に遅延の兆候を気づかせるきっかけとして機能しているかを確認する。
2. **行動修正の誘発：**遅延を示す指標が提示された際に，利用者の実際の作業量（pace7d）が増加するなど，行動変化が生じているかを検証する。
3. **進捗状態の理解促進：**SPI の折れ線グラフや EAC の時系列可視化が，利用者に自身のタスク状況を理解させる助けとなっているかを評価する。
4. **記録習慣の形成：**実績入力リマインド通知や日次ログ可視化が，記録忘れの防止や継続的な自己調整行動の維持に寄与しているかを確認する。

なお，見積時間（E）と実績時間（A）の差異（見積誤差）は，本研究の主目的ではないため，補助的な観点として扱う。実験全体としては，上記 4 点に基づきログデータおよびアンケート結果を分析し，提案モデルの有効性を評価する。

5.2 方法と対象

被験者： 大学の学習・制作タスクを日常的に記録する学生 3 名に協力を依頼した。
実施期間： 2 週間（第 1 週：導入期、第 2 週：定着期）
操作フロー：

1. 実験開始時にユーザーは複数のタスクを登録（E と締切を入力）
2. 毎日、実績を ProgressEntry 画面から記録。リマインド通知が夜に送信される。
3. 分析画面を定期的に開き、SPI・EAC・pace7d を確認可能な環境で運用

4. タスク完了時、E と A が自動で比較され、見積誤差が記録に残る
5. 2 週間の実験終了後、CSV 形式でタスクデータ・記録データ・通知ログを回収
6. ユーザーにアンケートを実施し、指標の使用実感・記録習慣の意識変化を聴取

5.3 ログの分析方針と可視化項目

- 完了タスクについて、A と E の差（誤差率）を折れ線プロットし、週ごとの精度変化を比較（図??）
- 各タスクの SPI の推移をグラフ化し、乱高下の有無や安定性を評価（図??）
- pace7d の変化と EAC 予測日のブレ幅を可視化し、「早めに取り組めたか」「締切超過リスクが低減したか」を分析
- 通知の送信タイミングと記録ログの時刻を突き合わせ、反応率を日別に集計（棒グラフ）

5.4 リスクと限界への備え

本実験は少人数・短期間であるため、次のような制約が存在する：

- ユーザーが SPI や EAC の意味を十分に理解していなければ、指標は活用されない可能性がある
- 通知への反応率や記録頻度には、生活スタイルや性格による個人差が大きく影響する
- 観察される改善傾向（見積精度や安定性）が、アプリの効果によるものか自然変化かを分離しにくい

こうした課題への対応として、アンケートで指標理解度や操作実感を尋ね、結果との整合性を分析する。また、今後は従来型 ToDo（SPI/EAC なし）との比較実験や、スマホ通知の精緻化による行動誘導の効果検証も視野に入れている。

6 結果・考察

7 まとめ

参考文献

[1] 野上 俊一, 生田 淳一, 丸野 俊一,“テスト勉強の学習計画と実際の学習活動とのズレに対する認識”, 日本教育工学会論文誌, 28(suppl), pp. 173–176, 2005.

[2] Project Management Institute, “A Guide to the Project Management Body of Knowledge (PMBOK® Guide) – Seventh Edition”, Project Management Institute, 発行年:2021.

[3] NASA, “Earned Value Management (EVM) Implementation Handbook”, NASA, 発行年:2019.

[4] U.S. Department of Defense, “Earned Value Management System (EVMS) Guidelines”, DoD, 発行年:2018.